Hochschule Ravensburg-Weingarten
University of Applied Sciences

PROJECT REPORT

Computer Vision

Task 2: Object Detection and Depth Estimation

Submitted by :

Rajveersinh Narendrasinh Suratiya - 10544165
Master in Electrical Engineering and Embedded Systems

Hardikkumar Kamrubhai Rathwa - 17744092
Masters in Mechatronics

Guided by :

Prof. Dr. Stefan Elser

January 10, 2025

Contents

1 Introduction **2**

2 Advanced Object Detection Techniques **2**
2.1 YOLO Methodology 2
2.2 Intersection Over Union (IoU) 2
2.3 Precision and Recall Calculation 4

3 Depth Estimation Methods **5**
3.1 Camera Calibration and Intrinsic Matrix 5
3.2 Converting 2D to 3D Coordinates 5
3.3 Calculating Real-World Distances 6

4 Validation and Evaluation of Results **7**
4.1 Factors Contributing to Discrepancies 8
4.2 Ananlysis of Specific Cases 9

5 Conclusion **11**

6 References **12**

List of Figures

1 filtered out non-car objects based on an IoU threshold 3
2 Evaluated images with precision and Recall value 4
3 Three coordinate systems used in the camera calibration. this link. 5
4 Distance measurement 6
5 Depth Estimation 7
6 No Cars Detection 7
7 Graph between YOLO distances with the Actual distances 8
8 Overlapping Objects 9
9 Topology of the road 9
10 Detected Midpoint 10
11 Complex Background 10

1 Introduction

This project focuses on detecting objects and estimating their distance from the camera using computer vision techniques. For object detection, we use the YOLO (You Only Look Once) algorithm, known for its speed and accuracy in identifying objects in real time. Depth estimation is incorporated to calculate the distance of these detected objects from the camera, which is critical for applications like autonomous navigation and robotics.

The primary aim is to create a system that not only identifies objects in a scene but also provides precise distance measurements, enabling better spatial understanding and interaction. This report covers the methods, implementation, and results of using YOLO for object detection and depth estimation.

2 Advanced Object Detection Techniques

2.1 YOLO Methodology

The **YOLO (You Only Look Once)** algorithm segments the input image into an S×S grid and predicts bounding boxes for each cell. The **YOLOv8x model** was employed for object detection, with additional steps implemented for precise car measurements.

The process starts by loading a pre-trained YOLOv8x model from the ultralytics library in Python for object detection, benefiting from its comprehensive training on various image datasets. Next, the model performs object detection by predicting bounding boxes,confidence scores, and class probabilities for each cell in the image grid. After detection, the algorithm extracts bounding box coordinates and class labels from the model predictions to for analysis.

2.2 Intersection Over Union (IoU)

The performance of the object detection system is evaluated using Intersection over Union (IoU). IoU is a crucial metric for assessing the accuracy of the predicted bounding boxes. It calculates the overlap between the predicted bounding box and the ground truth box, defined as the ratio of the intersection area to the union area.

```
def calculate_iou(box1, box2):
    x1 = max(box1[0], box2[0])
    y1 = max(box1[1], box2[1])
    x2 = min(box1[2], box2[2])
    y2 = min(box1[3], box2[3])
    intersection_area = max(0, x2 - x1 + 1) * max(0, y2 - y1 + 1)
    area_box1 = (box1[2] - box1[0] + 1) * (box1[3] - box1[1] + 1)
    area_box2 = (box2[2] - box2[0] + 1) * (box2[3] - box2[1] + 1)

    union_area = area_box1 + area_box2 - intersection_area
    iou = intersection_area / union_area
    return iou
```

The code determines the intersection coordinates of the two bounding boxes, computes their areas, and calculates the IoU by dividing the intersection area by the union area. The mathematical equation for two bounding boxes B1 and B2 with coordinates (x1,y1,x2,y2) is as follows:

$$\text{IoU}(B_1, B_2) = \frac{\text{Area of Overlap}}{\text{Area of Union}} \tag{1}$$

Where:

- **The Area of Overlap:** The area shared by both bounding boxes.
- **The Area of Union:** The total area covered by both bounding boxes, including the overlap.

The IoU calculation is essential for assessing the overlap between predicted and ground truth bounding boxes, allowing for the filtering of detections using a predetermined threshold.

```
# Filter out detections based on IoU with ground truth boxes
filtered_boxes = []
for box, conf in zip(boxes, confidences):
    x1, y1, x2, y2 = box
    detected_box = [x1, y1, x2, y2]
    max_iou = max([calculate_iou(detected_box, gt_box) for gt_box in gt_boxes])
    if max_iou >= 0.6:
        filtered_boxes.append(box)
```

The code filters out non-car objects based on an IoU threshold of 0.6,with significant overlap with the ground truth boxes are retained. Filtered images displayed below :

Figure 1: filtered out non-car objects based on an IoU threshold

2.3 Precision and Recall Calculation

To assess the performance of the model across entire scenes, we further computed precision and recall.A threshold of 0.6 was employed to determine whether a detection is classified as a true positive. A detection was considered valid (true positive) if its IoU exceeded the predefined threshold (0.6). Unmatched ground truth boxes were categorized as false negatives, while unmatched predictions were treated as false positives.

- **Precision** : This metric measures the proportion of true positive detections out of all predicted bounding boxes. A high precision value indicates that most detected objects are accurate.

- **Recall** : This metric reflects the proportion of true positive detections out of all ground truth objects. A high recall value signifies that the model detects most of the objects in the scene.

```
# Function to calculate Rc and Pc based on the formulas
def calculate_rc_pc(tp_c, fn_c, fp_c):
    # Rc = TPc / (TPc + FN(c))
    Rc = tp_c / (tp_c + fn_c) if (tp_c + fn_c) != 0 else 0
    # Pc = TPc / (TPc + FP(c))
    Pc = tp_c / (tp_c + fp_c) if (tp_c + fp_c) != 0 else 0
    return Rc, Pc
```

Above code used for the Precision and Recall calculation of images.

In the output images, these values are displayed on the right side of each frame to give a quick summary of the detection performance.

- **Evaluation Results** : All the evaluated scenes, including detailed annotations of IoU, precision, and recall, have been compiled for review.

For access to the complete set of annotated images and detailed results, please use the following link: **Evaluation Results with IOU,PC and RC**.

Figure 2: Evaluated images with precision and Recall value

3 Depth Estimation Methods

3.1 Camera Calibration and Intrinsic Matrix

Camera calibration involves determining the camera's intrinsic parameters. The calibration matrix is typically obtained using calibration techniques like Zhang's method or using calibration patterns like a chessboard.

In the KITTI Selection dataset, the camera intrinsic matrix K is as follows:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- f_x and f_y are the focal lengths of the camera in the x and y directions, respectively.
- (c_x, c_y) represents the principal point, which is the point where the optical axis intersects the image plane.
- s is the skew factor, typically 0 in most cameras, indicating any misalignment between the x and y axes.

3.2 Converting 2D to 3D Coordinates

Convert pixel coordinates to 3D world coordinates using the inverse of the intrinsic matrix. Given the camera calibration matrix K, the conversion from 2D pixel coordinates to 3D world coordinates involves reversing the projection process. The homogeneous equation

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}$$

can be solved to obtain the 3D coordinates (X, Y, Z) using the inverse of the calibration matrix K^{-1}.

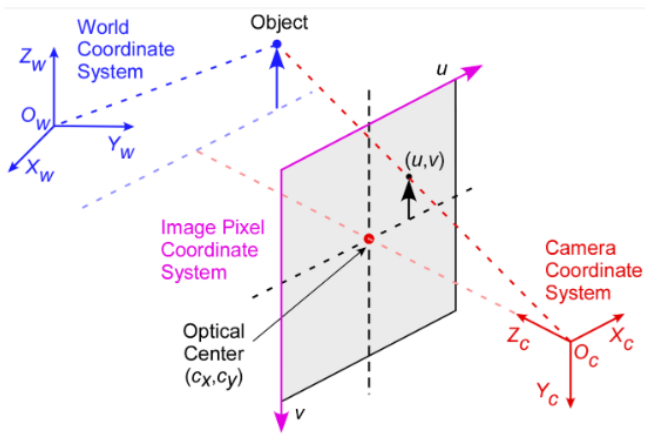

Figure 3: Three coordinate systems used in the camera calibration. this link.

```python
# Function to convert pixel coordinates to 3D world coordinates
def convert_to_3D(pixel_coordinate, K_inv, scalar):
    pixel_homogeneous = np.append(pixel_coordinate, 1).reshape(3, 1)
    ray_direction = np.dot(K_inv, pixel_homogeneous)
    point_3D = scalar * ray_direction.flatten()
    return point_3D
```

The "convert to 3D" function transforms 2D pixel coordinates into 3D world coordinates using homogeneous representations and the inverse camera intrinsic matrix. By computing the ray direction and scaling it by a scalar value, it accurately determines the position of objects in 3D space.

3.3 Calculating Real-World Distances

The camera is positioned in a height of 1.65 m from the driving plane.The camera is also aligned to the horizon. The distance of an object from the camera can be estimated using the 3D coordinates of its bottom center. Euclidean Distance formula is used to measure the distance:

$$\text{Distance} = \sqrt{X_c^2 + Y_c^2 + Z_c^2}$$

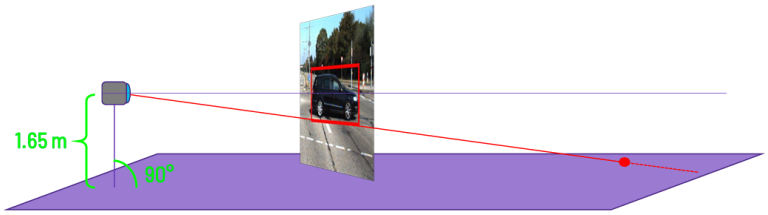

Figure 4: Distance measurement

```python
# Function to calculate real-world distance using 3D conversion
def calculate_real_world_distance_3D_conversion(bottom_center, K, camera_height):
    fx, fy, cx, cy = K[0, 0], K[1, 1], K[0, 2], K[1, 2]
    K_inv = np.linalg.inv(K)
    real_world_Z = camera_height * fy / (bottom_center[1] - cy)
    bottom_center_3D = convert_to_3D(bottom_center, K_inv, real_world_Z)
    distance = np.linalg.norm(bottom_center_3D)
    return distance
```

Explaination : The `calculate_real_world_distance_3D_conversion` function estimates the real-world distance from the camera to an object using its pixel coordinates in the image. It utilizes the camera's intrinsic matrix (K) to determine the object's position in 3D space.

The function calculates the depth (Z_c) of the object based on the camera's height and the vertical pixel coordinate of the object's bottom center. Using the inverse of the intrinsic matrix, the pixel coordinates are converted into 3D real-world coordinates.

Finally, the Euclidean distance is computed to provide the straight-line distance from the camera to the object.

Figure 5: Depth Estimation

The detailed distance calculations for each image are compiled and can be explored via the following link: **Depth Estimation Results**

Additionally, there are images where no cars are detected, thus these images are ignored during both the detection and depth estimation processes.

Figure 6: No Cars Detection

4 Validation and Evaluation of Results

To evaluate the accuracy of distance estimation using the YOLO model, a scatter plot was utilized to compare the YOLO distances with the actual distances. The red dashed line in the plot represents the ideal scenario where YOLO estimates perfectly match the real distances. The majority of the blue dots cluster closely around this Ideal Line, indicating that the YOLO model's distance estimates are generally accurate. However, discrepancies become noticeable, especially with longer distances.

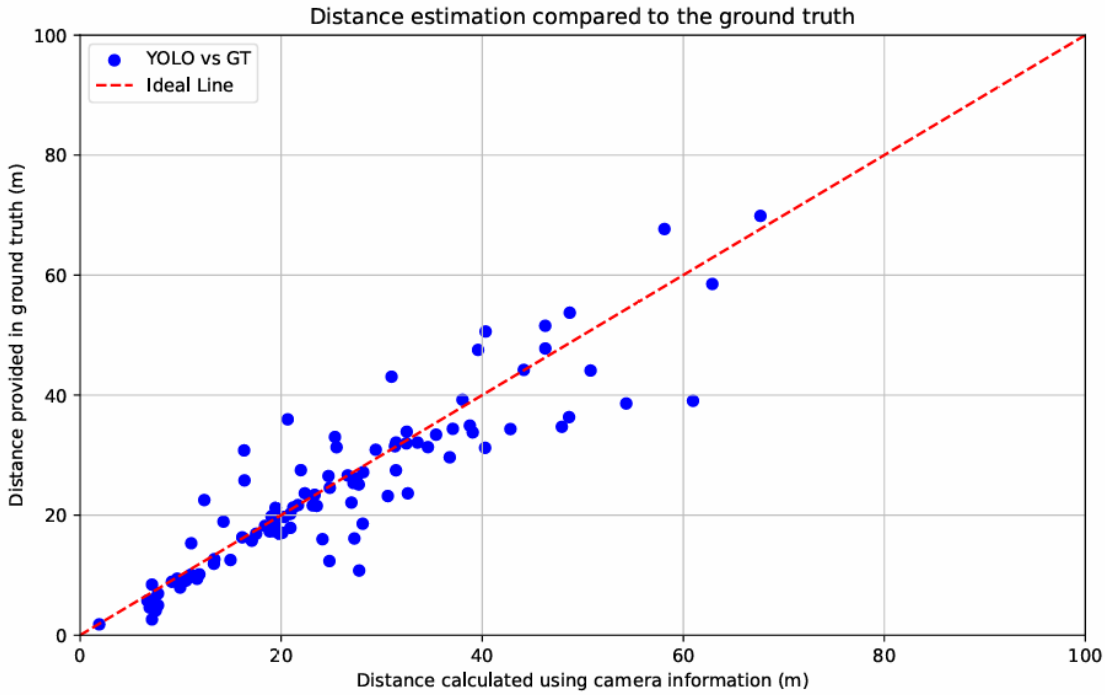

Figure 7: Graph between YOLO distances with the Actual distances

4.1 Factors Contributing to Discrepancies

- **Model Limitations:** The YOLO model may face challenges in accurately detecting objects or estimating distances over greater distances. Perspective effects can alter the aspect ratios of distant objects, making bounding box predictions more difficult, particularly for objects with extreme aspect ratios.

- **Object Size:** As the distance between the object and the camera increases, the object appears smaller in the image, occupying fewer pixels. This reduction in size makes it difficult for the `YOLOv8x.pt` model to accurately detect and localize objects. Objects that are smaller than 16x16 or 32x32 pixels in the image can be particularly challenging for the model to detect, leading to inaccuracies in distance estimation.

- **Camera Limitations:** The camera used for image capture may have limitations in resolution, lens quality, and performance under various environmental conditions such as poor lighting or obstructions. These factors can impact the clarity and detail of images captured at longer distances, thus affecting the model's ability to accurately estimate distances.

- **Environmental Factors:** Conditions such as lighting, weather, and the presence of obstructions can influence the quality of the images and the accuracy of the object detection and depth estimation. Poor lighting or occlusions can make it difficult for the model to detect objects and estimate distances accurately.

4.2 Ananlysis of Specific Cases

In some images, the estimations were significantly different from the ground truth. Here are few examples and potential reasons for these discrepancies:

Figure 8: Overlapping Objects

- **Overlapping Objects:** In above images, multiple objects were overlapping, causing the YOLO model to struggle with distinguishing the boundaries between them. it's difficult to determine which parts of the bounding box correspond to which vehicle.This overlap likely led to incorrect bounding box predictions, resulting in inaccurate distance estimations.

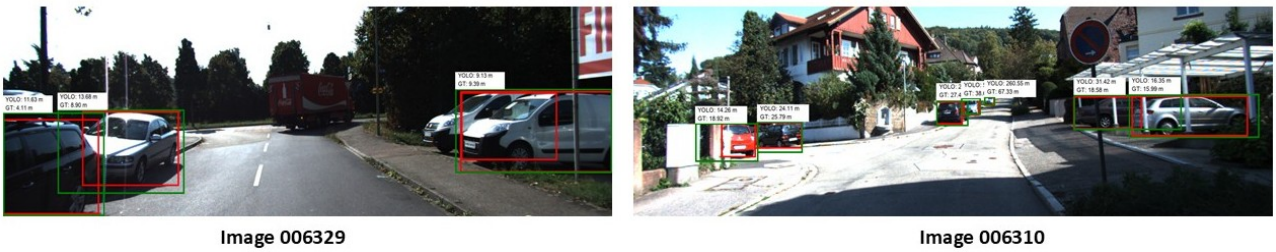

Figure 9: Topology of the road

- **Topology of the road :** In the images above, the road's slope and curvature distort the appearance of the objects. On curved roads, vehicles can appear farther or closer depending on the perspective, leading to errors in distance estimation. The camera's angle relative to the curve also causes the objects to seem inaccurately sized, complicating the distance calculation estimations.

Image 006042 Image 006374

Figure 10: Detected Midpoint

- **Detected Midpoint:** Mention images, the midpoint of the bounding box, often used for distance estimation, may not always align with the actual center of the object, especially when the object is irregularly shaped or partially visible. This misalignment leads to errors in distance calculation. if the object is positioned off-center in the image or at an angle, the detected midpoint might not accurately reflect its true 3D position, further affecting the distance estimation.

Image 006037 Image 00315

Figure 11: Complex Background

- **Complex Background:** An image with a complex and cluttered background caused the model to misidentify parts of the background as part of the object. This misidentification affected the bounding box prediction and thus the distance estimation.

Overall, while the YOLO model demonstrates a high level of accuracy in distance estimation, particularly for closer objects, these factors collectively contribute to the observed discrepancies at longer distances. Addressing these limitations could involve improving model training with more diverse and higher-resolution datasets, enhancing camera capabilities, and refining detection algorithms to better handle perspective effects.

5 Conclusion

This project successfully explored the integration of object detection and distance estimation, understanding scenes in various computer vision applications. This project demonstrated the use of the YOLO model for object detection in conjunction with monocular depth estimation techniques for distance calculation. The results indicated that the YOLO model is capable of accurately detecting objects and estimating distances, particularly for closer objects. However, Distance estimation is affected by factors such as complex backgrounds, detected midpoints, road topology including slopes and curves, and overlapping objects. These elements contribute to the overall challenges in obtaining accurate measurements in dynamic environments.

Key learnings from this project include the critical importance of model robustness and adaptability, as well as the significant impact of environmental factors on detection accuracy. Overall, this project highlights the potential and effectiveness of combining object detection and depth estimation techniques in computer vision.

6 References

1. YOLO Real-time object detection and image segmentation model
2. https://pypi.org/project/numpy/
3. Moodle chapter 5, chapter 6, chapter 7, and chapter 8